

CRAFT — Pitch Package

Audited retrieval-augmented AI assistant for CSA mission engineering and finance.

1. System Context (L1)

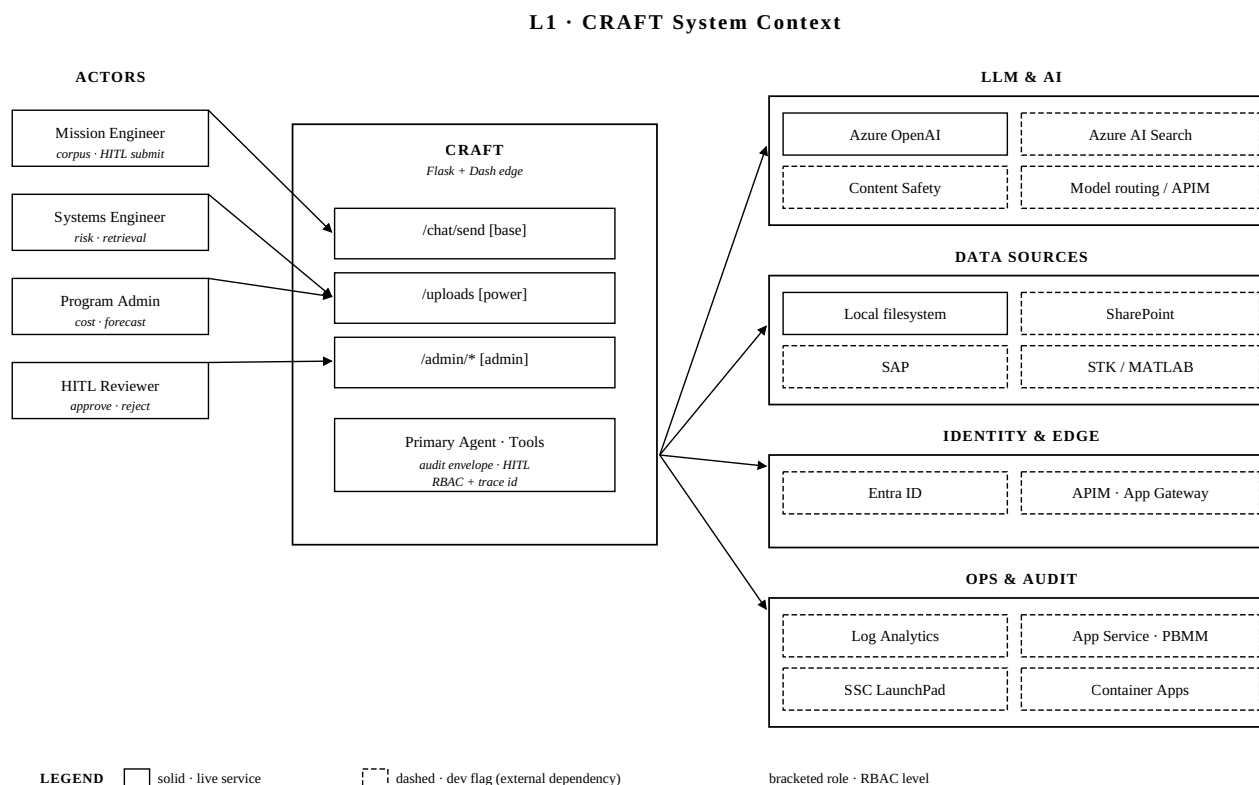


Figure 1 · Actors, external systems, and the CRAFT boundary.

Four actor roles reach CRAFT through a single Flask + Dash edge. Inbound traffic is role-gated at /chat/send, /uploads, and /admin/*. Outbound traffic fans to four control planes: LLM & AI (Azure OpenAI, Azure AI Search, Content Safety), Data Sources (local FS, SharePoint, SAP, STK/MATLAB), Identity & Edge (Entra ID, APIM, App Gateway), and Operations & Audit (Log Analytics, App Service PBMM, LaunchPad, Container Apps). Dashed outlines in the diagram mark services staged behind IT or procurement gates — the integration seams are production, the backing services arrive on IT's cadence.

2. Containers (L2)

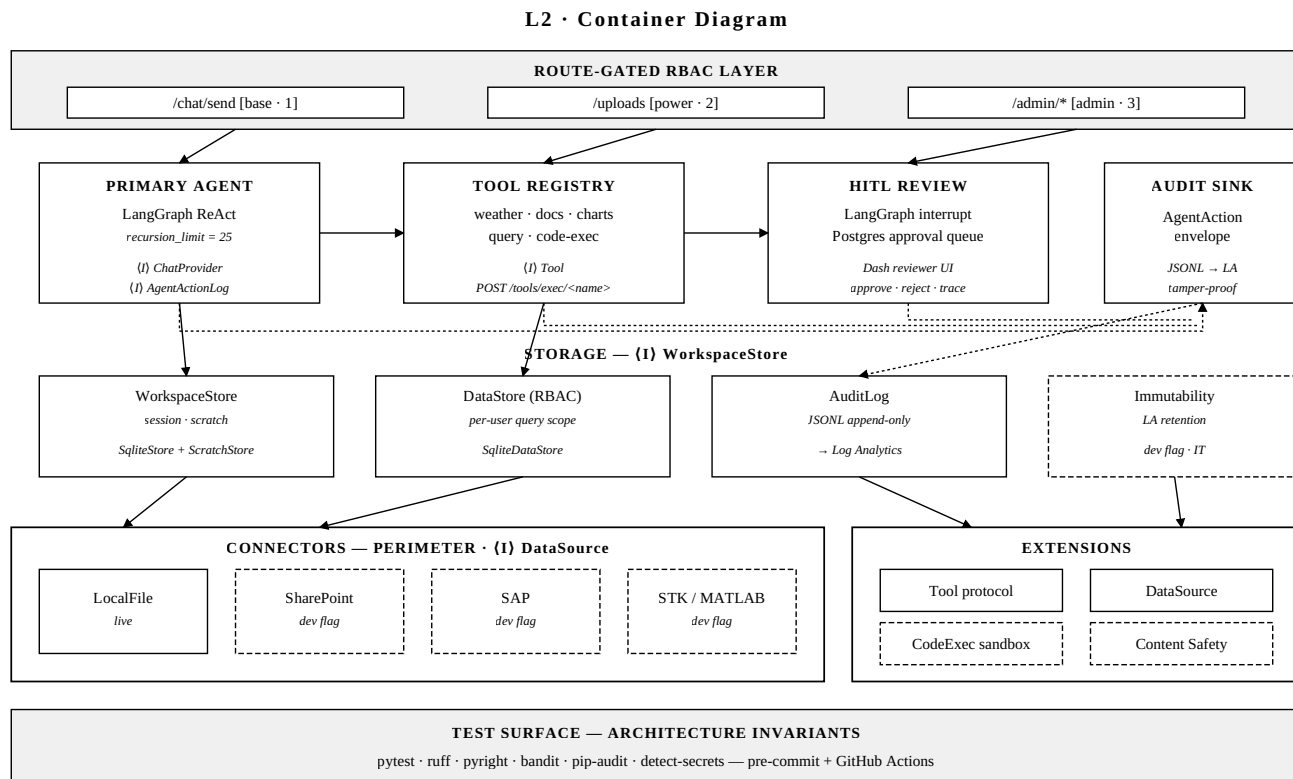


Figure 2 · Internal containers, interfaces, and the RBAC + audit perimeters.

Requests enter the route-gated RBAC layer and are dispatched to the Primary Agent, which drives a Tool Registry over the Tool protocol and reads/writes through WorkspaceStore, DataStore, and AuditLog. Every tool and LLM turn emits an AgentAction envelope to the Audit Sink and from there to Log Analytics. Perimeter Connectors (LocalFile, SharePoint, SAP, STK/MATLAB) sit outside the audit boundary and are reached only through DataSource adapters. The test surface validates architectural invariants (route rules, role levels, protocol contracts) on every commit.

3. Internals (L3)

L3 · Internal Clusters

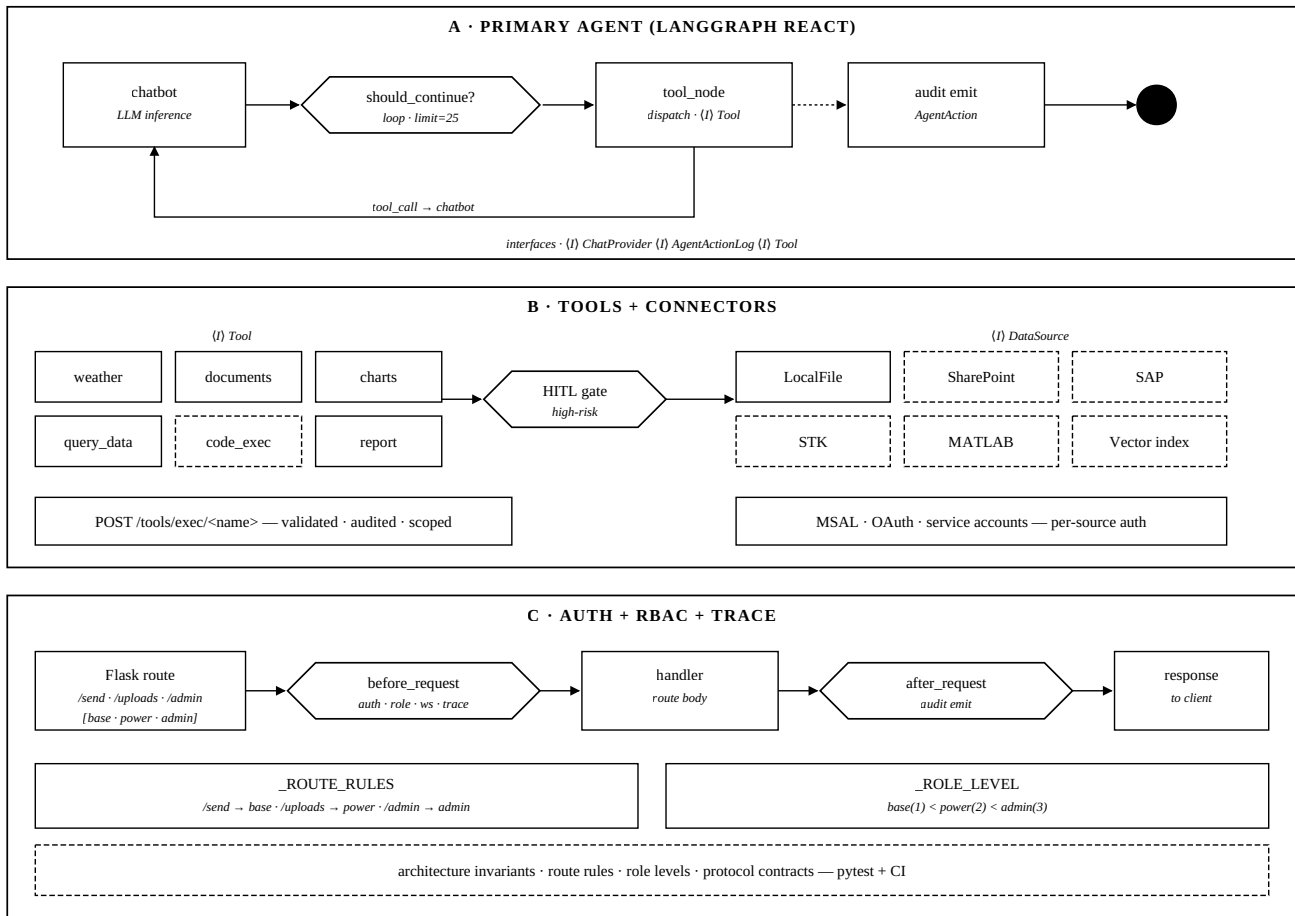


Figure 3 · Primary agent loop, tool/connector cluster, and auth cluster.

Agent cluster. The LangGraph chatbot node calls the tool node, loops on `_should_continue` with `recursion_limit=25`, and funnels `llm_inference` and `tool_call` into the audit sink via `AgentActionLog`. **Tools + connectors cluster.** The `Tool` protocol groups weather, document, chart, query, and code-exec tools; the `DataSource` protocol groups LocalFile, SharePoint, SAP, and STK adapters; a HITL gate interrupts high-risk calls before they reach `/tools/exec/<name>`. **Auth cluster.** Every public route runs through `before_request` (auth, role, workspace, trace id), the handler, then `after_request` (audit emit). `_ROUTE_RULES` maps `/send→base`, `/uploads→power`, `/admin→admin`; `_ROLE_LEVEL` enforces `base(1) < power(2) < admin(3)`. Architecture invariants are checked by the audit-green test surface.

4. Use Cases (One-Sheet)

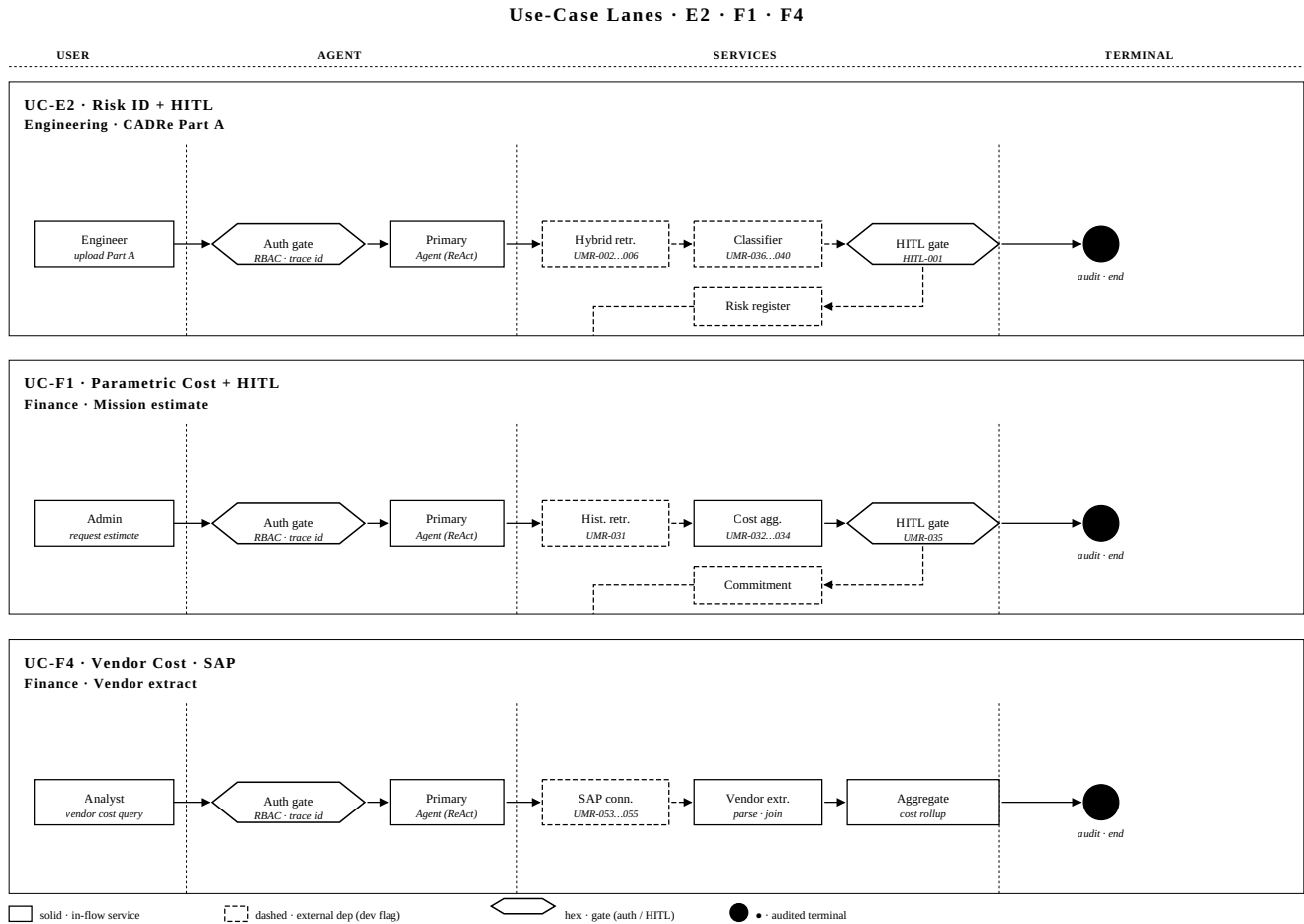


Figure 4 · Three use-case lanes laid out left-to-right with UC IDs and swim-lane dividers.

Each lane runs User → Agent → Services → Terminal left to right. Dashed nodes mark steps whose backing service is still an IT or data-governance dependency — the control flow and audit envelope through the lane are identical whether the step runs against the production service or its scratch adapter.

- **UC-E2 · Risk ID + HITL (Engineering).** Engineer uploads CADRe Part A → auth gate → hybrid retrieval over corpus [UMR-002...006] → risk classifier [UMR-036...040] → HITL gate [HITL-001] → risk register write → audit → terminal.
- **UC-F1 · Parametric Cost + HITL (Finance).** Admin requests estimate → auth gate → historical retrieval [UMR-031] → cost aggregator [UMR-032...034] → HITL gate [UMR-035] → commitment write → audit → terminal.
- **UC-F4 · Vendor Cost via SAP (Finance).** Analyst queries vendor cost → auth gate → SAP connector [UMR-053...055] → vendor extract → aggregate → audit → terminal.

Other mission flows (budget scenarios, anomaly triage, bilingual reports, cross-mission comparison) attach as new Tool + DataSource pairs along the same lanes.

5. Requirements → Design

CAPABILITY	REQUIREMENTS	DESIGN SEAM	USE CASES
Primary agent	UMR-001/011/013/016/018/019/094/095, AAR-001/003/006, ASG-005	LangGraph ReAct · ChatProvider · recursion_limit=25	E2, F1, F4
Retrieval + RAG	UMR-002/003/004/005/006/009/010/060/067, ARR-002–007	Azure AI Search (hybrid vector + BM25) · DataSource + Tool	E2, F1
Audit + traceability	UMR-015/027/045/061/062/093, AAR-005, ASG-003	AgentAction envelope · AgentActionLog · JSONL → Log Analytics	E2, F1, F4
HITL approvals	UMR-021–026/035/039/044/049, HITL-001–007, ASG-002	LangGraph interrupt · Postgres queue · Dash review UI	E2, F1
RBAC + data scoping	UMR-012/017/058/065/066, AAR-002/007	_ROUTE_RULES + _ROLE_LEVEL · SqliteDataStore · QueryTool.set_user	E2, F1, F4
Session + workspace	UMR-014/092/095	WorkspaceStore → SqliteStore + SessionScratchStore	all
Guardrails + containment	UMR-020/029/030, ASG-001/004/011	Azure Content Safety · APIM rate limit · Container Apps sandbox	all
Connectors + external	UMR-051–055	DataSource + MSAL · Flask REST · SAP / STK / MATLAB adapters	F4
Domain capabilities	UMR-007/008/031–034/036–038/040–043/046–048/050/075, ARR-001	New Tool instances over existing seams	E2, F1
Deployment, SLA	UMR-028/056/057/059/064/069/070/074/085, ASG-006	Azure App Service PBMM · APIM + WAF · SSC LaunchPad HA · git config	all

6. Development Flags & System-Health Tooling

Development flags surface the external dependencies still in flight; health tooling is run on every commit and in CI.

ITEM	KIND	OWNER / TOOL	NOTES
Azure AI Search	Dev flag	IT	Hybrid retrieval + citations. Seam: DataSource + HybridRetrievalTool.
Log Analytics + immutability	Dev flag	IT	Tamper-proof audit sink. Seam: AgentActionLog.
Content Safety + Container Apps	Dev flag	Procurement	Guardrails + sandboxed code exec. Seam: CodeExecTool.
Entra ID + SharePoint / SAP	Dev flag	IT + Finance	CSA corpus + cost data. Seam: DataSource.
CSA risk taxonomy	Dev flag	Domain	Risk ID + HITL classification.
Historical mission DB	Dev flag	Finance	Parametric cost estimation.
ruff	Health	pre-commit + CI	Lint (flake8 + isort + pyupgrade).
pyright	Health	pre-commit + CI	Strict type checking.
pytest + coverage	Health	pre-commit + CI	156 tests, 91% coverage.
bandit	Health	pre-commit	Security scan.
pip-audit	Health	pre-commit	Dependency CVE audit.
detect-secrets	Health	pre-commit	No credentials in git.